

# Architecture design patterns that support cost optimization

When you design workload architectures, you should use industry patterns that address common challenges. Patterns can help you make intentional tradeoffs within workloads and optimize for your desired outcome. They can also help mitigate risks that originate from specific problems, which can affect reliability, security, performance, and operations. If not mitigated, risks will eventually increase costs. These patterns are backed by real-world experience, are designed for cloud scale and operating models, and are inherently vendor agnostic. Using well-known patterns as a way to standardize your workload design is a component of operational excellence.

Many design patterns directly support one or more architecture pillars. Design patterns that support the Cost Optimization pillar align with implementing favorable billing models, reducing overprovisioning, changing scaling dimensions, and maximizing value during migrations.

The following table summarizes Architecture design patterns that support the goals of cost optimization.

 Expand table

| Pattern                                        | Summary                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Claim Check</a>                    | Separates data from the messaging flow, providing a way to separately retrieve the data related to a message. Messaging systems often impose limits on message size, and increased size limits is often a premium feature. Reducing the size of message bodies might enable you to use a cheaper messaging solution.                                                                                      |
| <a href="#">Competing Consumers</a>            | Applies distributed and concurrent processing to efficiently handle items in a queue. This pattern can help you optimize costs by enabling scaling that's based on queue depth, down to zero when the queue is empty. It can also optimize costs by enabling you to limit the maximum number of concurrent consumer instances.                                                                            |
| <a href="#">Compute Resource Consolidation</a> | Optimizes and consolidates compute resources by increasing density. This pattern combines multiple applications or components of a workload on a shared infrastructure. Doing so maximizes the utilization of computing resources by avoiding unused provisioned capacity via aggregation of components or even whole workloads on a pooled infrastructure. Container orchestrators are a common example. |
| <a href="#">Gateway Offloading</a>             | Offloads request processing to a gateway device before and after forwarding the request to a backend node. Adding an offloading gateway into the request process enables you to redirect costs from resources that would be spent per-                                                                                                                                                                    |

| Pattern                   | Summary                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | node into the gateway implementation. Costs in the centralized processing model are frequently lower than those of the distributed model.                                                                                                                                                                                                                                                                                                                           |
| Messaging Bridge          | Provides an intermediary to enable communication between messaging systems that are otherwise incompatible because of protocol or format. This intermediary can increase the longevity of your existing system while still allowing interoperability with systems that use a different messaging or eventing technology.                                                                                                                                            |
| Publisher/Subscriber      | Decouples components of an architecture by replacing direct client-to-service or client-to-services communication with communication by using an intermediate message broker or event bus. This design can enable an event-driven approach in your architecture, which couples well with consumption-based billing to avoid overprovisioning.                                                                                                                       |
| Queue-Based Load Leveling | Controls the level of incoming requests or tasks by buffering them in a queue and letting the queue processor handle them at a controlled pace. Because load processing is decoupled from the request or task intake, you can use this approach to reduce the need to overprovision resources to handle peak load.                                                                                                                                                  |
| Sharding                  | Directs load to a specific logical destination to handle the specific request, enabling colocation for optimization. A system that implements shards often benefits from using multiple instances of less expensive compute or storage resources rather than a single more expensive resource. In many cases, this configuration can save you money.                                                                                                                |
| Static Content Hosting    | Optimizes the delivery of static content to workload clients by using a hosting platform that's designed for that purpose. Dynamic application hosts are usually more expensive than static hosts because dynamic hosts can run your coded business logic. Using an application platform to deliver static content isn't cost-effective.                                                                                                                            |
| Strangler Fig             | Provides an approach for systematically replacing the components of a running system with new components, often during a migration or modernization of the system. The goal of this approach is to maximize the use of existing investments in the currently running system while modernizing incrementally. It enables you to perform high-ROI replacements before low-ROI replacements.                                                                           |
| Throttling                | Imposes limits on the rate or throughput of incoming requests to a resource or component. The limits can inform cost modeling and can even be directly tied to the business model of your application. They also put clear upper bounds on utilization, which can be factored into resource sizing.                                                                                                                                                                 |
| Valet Key                 | Grants security-restricted access to a resource without using an intermediary resource to proxy the access. This design offloads processing as an exclusive relationship between the client and the resource without adding a component to directly handle all client requests. The benefit is most dramatic when client requests are frequent or large enough to require significant proxy resources or the proxy does not add value as being part of the request. |

# Next steps

Review the Architecture design patterns that support the other Azure Well-Architected Framework pillars:

- [Architecture design patterns that support reliability](#)
- [Architecture design patterns that support security](#)
- [Architecture design patterns that support operational excellence](#)
- [Architecture design patterns that support performance efficiency](#)

---

Last updated on 08/29/2025